

Engineering Standards & Guidelines

Technical Standards, Code Quality, API Design, and Deployment Practices
Engineering Department

Document ID: TVSC-ENG-003 | Version 4.1 | Last Updated: February 2026

1. Approved Technology Stack

TechViet maintains a curated set of approved technologies to ensure security, maintainability, and knowledge sharing across teams. Using technologies outside this list requires an Architecture Decision Record (ADR) approved by the Architecture Review Board (ARB), which meets bi-weekly. The ARB consists of the CTO, VP Engineering, Principal Engineers, and the CISO.

Layer	Primary Choice	Approved Alternatives	Decision Rationale
Frontend Web	React 18 + TypeScript 5.x	Next.js 14 (SSR needs), Vue 3 (legacy projects)	React ecosystem maturity; TypeScript mandatory for all new frontend code since 2024
Backend API	Python 3.12 + FastAPI 0.110+	Go 1.22 (high-perf services), Node.js 20 (BFF)	FastAPI for ML-heavy services and rapid prototyping; Go for latency-critical microservices
Mobile	React Native 0.73+	Flutter 3.19+ (approved for new projects Q1 2026)	React Native for code sharing with web; Flutter approved after successful ShopSmart pilot
Relational DB	PostgreSQL 16	MySQL 8.0 (client requirement only)	PostgreSQL for JSON support, extensions (pgvector, PostGIS), and performance at scale
NoSQL	MongoDB 7.0	Redis 7.2 (cache/queue), DynamoDB (serverless)	MongoDB for document workloads; DynamoDB only for pure serverless architectures on AWS
Message Queue	Apache Kafka 3.7	RabbitMQ 3.13 (simple task queues)	Kafka for event streaming and audit logs; RabbitMQ for simple producer-consumer patterns
Search Engine	Elasticsearch 8.12	OpenSearch 2.12 (AWS-native)	Elasticsearch default; OpenSearch when client requires AWS-native and avoids Elastic licensing
Vector Database	Qdrant 1.8 (self-hosted)	Pinecone (prototyping), pgvector (small datasets <100K vectors)	Qdrant selected after 4-week PoC (see ADR-001); best cost-performance for our workloads
Containers	Docker + Kubernetes (EKS 1.29)	ECS Fargate (simple services)	K8s for all production workloads; ECS only for isolated scheduled tasks
CI/CD	GitHub Actions	None (Jenkins being decommissioned Q2 2026)	Standardization decision (see ADR-003); shared workflow templates in techviet/ci-templates
IaC	Terraform 1.7+ (OpenTofu compatible)	AWS CDK (complex AWS-native infra)	Terraform default for multi-cloud compatibility; CDK approved for pure-AWS projects
Monitoring	Datadog (APM + Logs + Infra)	Prometheus + Grafana (on-prem client deployments)	Datadog for SaaS simplicity; Prometheus stack when data residency prevents SaaS
LLM Provider	Anthropic Claude 3.5 Sonnet (primary)	OpenAI GPT-4o (fallback), local Llama 3 (offline/edge)	Claude for accuracy and safety; GPT-4o as redundancy; Llama for data-sovereign clients

2. Code Quality Standards

2.1 Pull Request and Code Review Requirements

Code review is one of our most important quality practices. Every change to a shared codebase must go through a pull request, regardless of the author's seniority. The following requirements apply to all repositories in the TechViet GitHub organization.

Requirement	Standard	Rationale
Minimum Reviewers	2 approvals for main/develop branches; 1 approval for feature branches	Two-person review catches more issues; single review OK for WIP branches to reduce friction
Review Turnaround	First review within 4 business hours; final approval within 24 hours	Fast feedback loops prevent context-switching costs; measured and reported in weekly eng metrics
Maximum PR Size	400 lines changed (excluding auto-generated files, tests, migrations)	Smaller PRs get better reviews. PRs >400 lines must be split or get explicit EM exception.
PR Description	Must include: What changed, Why (link to Jira ticket), How to test, Rollback plan	Structured descriptions enable async review and serve as documentation for future developers
CI Checks Required	Linting (ruff/eslint), unit tests, integration tests, type check, security scan (Snyk)	Automated checks must all pass before merge. Manual override requires 2 reviewer approvals.
Branch Naming	Format: {type}/{ticket}-{description}. Types: feat, fix, refactor, chore, docs, test	Consistent naming enables automated changelog generation and branch cleanup scripts
Commit Messages	Conventional Commits format. 72-char subject line. Body explains why, not what.	Enables automated release notes; enforced by commitlint in pre-commit hook
Stale PR Policy	PRs with no activity for 7 days are flagged. 14 days: auto-closed with comment.	Prevents PR backlog buildup; author is notified and can reopen with /reopen command

2.2 Testing Standards

Test Type	Coverage / Target	When Executed	Owner	Tooling
Unit Tests	≥80% line coverage (enforced by CI); ≥90% for core business logic modules	Every commit via GitHub Actions	Feature developer	pytest (Python), Jest (JS/TS), go test (Go)
Integration Tests	All API endpoints; all database operations; all external service integrations	Every PR; nightly full suite	Feature developer + QA	pytest + testcontainers; Playwright for API
End-to-End (E2E)	All critical user journeys (top 20 flows); top 5 error/edge-case scenarios	Nightly build; pre-release gate	QA Team	Playwright (migrating from Selenium; 60% done)
Performance Tests	P95 latency within SLA; no regressions >10% vs. baseline; memory leak detection	Weekly automated; pre-release mandatory	DevOps + QA	k6 (load); Vegeta (HTTP); custom profilers
Security Scans (SAST)	Zero critical/high findings; medium findings triaged within 5 business days	Every PR (Snyk); weekly full scan (Veracode)	Security Team + Developer	Snyk (PR), Veracode (weekly), CodeQL (GitHub)
Security Scans (DAST)	Zero critical findings on production-facing endpoints	Weekly against staging environment	Security Team	OWASP ZAP; Burp Suite (manual quarterly)

Test Type	Coverage / Target	When Executed	Owner	Tooling
Chaos Engineering	Production services survive: single node failure, network partition, dependency timeout	Monthly automated; quarterly game day	SRE Team	Litmus Chaos; custom failure injection

3. API Design Standards

All public and internal APIs must follow the TechViet API Design Guide. REST is the default protocol for client-facing APIs. gRPC is approved for internal service-to-service communication where low latency is critical. GraphQL is approved only for Backend-for-Frontend (BFF) layers.

Standard	Requirement	Example / Notes
Versioning	URL path versioning: /api/v1/resource. No breaking changes within a major version. Deprecation notice 6 months before sunset.	/api/v1/users, /api/v2/users. Header versioning explicitly not used.
Authentication	OAuth 2.0 + JWT (RS256) for external APIs. mTLS for internal service mesh (Istio). API keys for webhooks (HMAC-SHA256 signed).	JWT expiry: 15 min access, 7 day refresh. Rotate signing keys quarterly.
Rate Limiting	Default: 100 req/min per client (token bucket). Configurable per endpoint and client tier. Rate limit headers in every response.	X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset. 429 response with Retry-After header.
Pagination	Cursor-based for list endpoints (offset-based deprecated). Max page size: 100 items. Default: 20 items.	?cursor=abc123&limit=20. Response includes next_cursor and has_more fields.
Error Format	RFC 7807 Problem Details JSON. Include type, title, status, detail, instance, and correlation_id.	{"type": "/errors/not-found", "title": "User not found", "status": 404, "correlation_id": "abc-123"}
Idempotency	All POST/PUT/PATCH endpoints must support Idempotency-Key header. Keys valid for 24 hours.	Client generates UUID as idempotency key. Server returns cached response for duplicate keys.
Timeouts	Client timeout: 30s. Server processing: 60s. Long-running operations: return 202 Accepted + poll endpoint.	Async operations return {"status_url": "/api/v1/jobs/123"} for polling.
Documentation	OpenAPI 3.1 spec required. All endpoints documented with examples. Spec auto-generated from code annotations.	Swagger UI at /docs (dev/staging only). Redoc at /api-docs (all environments).

4. Deployment and Release Standards

Environment	Branch	Deploy Method	Approval Gate	Rollback Method	Monitoring
Development	feature/* branches	Auto-deploy on push (Argo CD)	None (automated)	Redeploy previous commit	Basic logging (stdout)
Staging	develop branch	Auto-deploy on merge to develop	1 code reviewer	Argo CD rollback (1-click)	Full Datadog APM + logs
UAT	release/* branches	Manual trigger (GitHub Actions)	QA Lead + Product Owner	Argo CD rollback	Full monitoring + Slack alerts
Production	main branch (tag-based)	Blue/green via Argo CD Rollouts	2 code reviewers + QA sign-off + EM approval	Automated rollback if error rate >1% in 10 min	Full monitoring + PagerDuty + status page

All production deployments must include a documented rollback plan. Canary deployments (10% traffic for 15 minutes) are required for changes affecting >10% of users. Feature flags via LaunchDarkly are mandatory for all new user-facing features — the flag must be in 'off' state at deploy time and enabled gradually after deploy verification.

Deploy windows: Production deploys are permitted Monday through Thursday, 9 AM — 4 PM ICT. No production deploys on Fridays, weekends, public holidays, or during client-designated freeze periods. Emergency hotfixes exempt from deploy window restrictions but require VP Engineering approval.